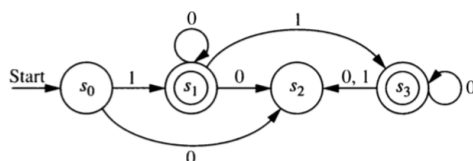
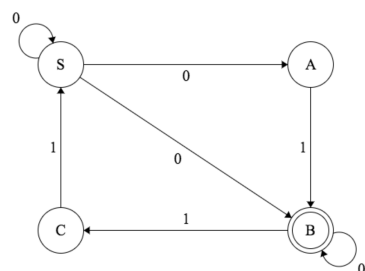
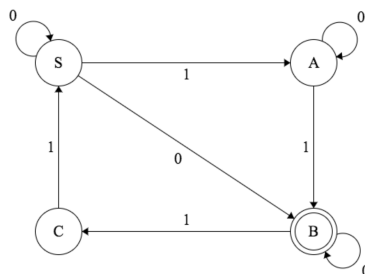


Regular Languages

Math 252 — Spring 2018

- Convert each of the following NFA's into an equivalent DFA. (Remember: equivalent means that they recognize the same language.) Do this using our general algorithm for this task. For the first two, the start state is S .



Operations on Languages

Since languages are sets, we can use set operations like $\cup$ and $\cap$ to create new languages. But there are some additional operations that we will use.

- concatenation of strings: xy is the concatenation of strings x and y (x followed by y)
- concatenation of sets: $AB = \{xy \mid x \in A \wedge y \in B\}$
- power: $A^0 = \emptyset$; $A^1 = A$; $A^{n+1} = A^n A$. (Power is iterated concatenation.)
- Kleene star: $A^* = \cup_{n=0}^{\infty} A^n$

- Which of these strings belong to $\{0,01\}^*$?
 - 01001
 - 10110
 - 00010
- Which of these strings belong to $\{011, 110, 11\}^*$?
 - 0101111
 - 0110110
 - 00010

Regular Expressions

Syntax

A regular expression over an input alphabet I is defined recursively.

- $\emptyset$ and λ and regular expressions.
- x is a regular expression for each $i \in I$.
- If A and B are regular expressions, then
 - A^* is a regular expression
 - $(A \cup B)$ is a regular expression
 - (AB) is a regular expression

Semantics

Each regular expression represents a language as follows.

- $\emptyset$ represents the empty language: $\emptyset$
- λ represents language that contains only the empty string: $\{\lambda\}$
- For each $x \in I$, x represents the language containing only x : $\{x\}$
- If A and B are regular expressions representing languages A and B , then
 - (AB) represents AB
 - $(A \cup B)$ represents $A \cup B$ [Note: This is often written $(A|B)$ in programming languages.]
 - A^* represents A^* [Note: This is often written as A^* in programming languages.]

The languages represented by regular expressions are called **regular languages** (how creative).

4. Describe in words the languages represented by each of these regular expressions.
 - a. 10^*
 - b. $(10)^*$
 - c. $0 \cup 01$
 - d. $0^* \cup 01$
 - e. $0(0 \cup 1)^*$
 - f. $(0 \cup 01)^*$
5. Can each of the languages above be recognized by an NFA? a DFA?
6. What would we need to do to show that every regular language can be recognized by an NFA? Give it a try and see how far you get. Which parts are easy? Which are trickier? Why?

Recognizing Regular Languages with NFAs

1. Show that the empty language ($\emptyset$) is recognized by an NFA.
2. Show that the language $\{\lambda\}$ is recognized by an NFA.
3. Show that if $a \in I$, then the language $\{a\}$ is recognized by an NFA.
4. Show that if A and B are each recognized by an NFA, then AB is, too.
[Hint: Let N_A and N_B be the NFAs that recognize A and B . How can you use them to build a new NFA that recognizes AB ?]
5. Show that if A and B are each recognized by an NFA, then $A \cup B$ is, too.
[Hint: Let N_A and N_B be the NFAs that recognize A and B . How can you use them to build a new NFA that recognizes $A \cup B$?]
6. Show that if A is recognized by an NFA, then A^* is, too.
[Hint: Let N_A be the NFA that recognizes A . How can you use it to build a new NFA that recognizes $A \cup B$?]
7. Explain how 1–6 above show that every regular language is recognized by an NFA.
8. Explain how 7 implies that every regular language is recognized by a DFA.
9. The method just outlined is automatic (you could fairly easily write a computer program to do the translation from regular expression to NFA), and it provides a proof that all regular languages can be recognized by an NFA. But it might produce an NFA with more states than is minimally required. Create an NFA that recognizes $\mathbf{1}^* \cup \mathbf{01}$. Do this two ways:
 - a. Following the steps outlined in 1–6.
 - b. Any way you like, but using fewer states than in part a.
 - c. What is the smallest (fewest states) NFA you can find that recognizes $\mathbf{1}^* \cup \mathbf{01}$?
10. Show that if A is recognized by an NFA, then the complement of A ($\overline{A} = A^C$) is too.
[Hint: There is an easy way and a hard way – use the easy way.]

Solution to problem 9 on previous page

